MODULE 03 · DOMAIN 1 - ARCHITECTURE BUILT FOR THE CLOUD

Containers and Orchestration at Scale

Pick the platform, pick what scales, and treat the image registry as part of your security boundary.

DOMAIN 1 of 5 EXAM WEIGHT 25% MODULE 3 of 52

admin@sthithapragna.com

Choosing a container platform is choosing **how much of the machine** you still own.

Who manages what

OPTION	AWS MANAGES	YOU STILL MANAGE
ECS on Fargate	Hosts, patching, capacity	Task size and scaling policy
ECS Managed Instances	EC2 provisioning, patching, scaling	Instance requirements, if any
ECS on EC2	Control plane only	AMIs, patching, capacity providers
EKS Auto Mode	Nodes, load balancers, storage, networking	Workloads, VPC, disruption budgets
EKS managed node groups	Node provisioning and updates on request	Instance choice, upgrades, add-ons
EKS on Fargate	Hosts per pod	Pod sizing; no DaemonSets or GPUs

ON THE EXAM

Two newer rows, ECS Managed Instances and EKS Auto Mode, sit between Fargate and self-managed hosts.

ECS or EKS?

CHOOSE ECS WHEN

AWS-native, least overhead

- No existing Kubernetes investment
- Simplest operational model
- Deep integration with AWS services
- Team does not want to run Kubernetes

CHOOSE EKS WHEN

Kubernetes is the requirement

- Existing Helm charts, operators, manifests
- Portability across clouds or on premises
- Kubernetes ecosystem tooling
- Same API on EKS Anywhere and in the cloud

THE TIEBREAKER

Kubernetes in the scenario means EKS. Without it, ECS is usually the lower-overhead answer.

Fargate or ECS Managed Instances?

CHOOSE FARGATE WHEN

No hosts at all

- Per-task vCPU and memory billing
- Each task isolated on its own host
- No GPU support
- Instance type is not your concern

CHOOSE MANAGED INSTANCES WHEN

Specific EC2, still managed

- GPU, network-optimised or burstable types
- AWS patches roughly every 14 days
- EC2 price plus a management fee
- Spot and Capacity Reservations supported

THE TIEBREAKER

Needs a GPU or a particular instance family, and nobody wants to patch hosts: Managed Instances.

EKS Auto Mode in one slide

- **Karpenter-based scaling** launches nodes for unschedulable pods and removes idle ones.
- **Load balancers, storage and pod networking** provisioned for you.
- **21-day maximum node lifetime** keeps nodes patched by replacement.
- **Disruption budgets still yours.** A blocking PDB can stall node updates.
- **Custom NodePools** for instance choice. Do not edit the defaults.

> ON THE EXAM
>
> Auto Mode is managed Karpenter plus managed add-ons. Kubernetes stays; node operations go.

Scaling happens at two layers

MECHANISM	SCALES	DRIVEN BY
Horizontal Pod Autoscaler	Pods	CPU, memory or custom metrics
Cluster Autoscaler	Node groups	Pending pods; one ASG per group
Karpenter / Auto Mode	Nodes, any suitable type	Pending pods; consolidates idle nodes
ECS service auto scaling	Tasks	Application Auto Scaling policies
ECS capacity providers	EC2 instances	Managed scaling from task demand
Lambda	Execution environments	Requests, within concurrency limits

ON THE EXAM

Pods or tasks scaling without capacity scaling gives you Pending work. Both layers need a mechanism.

Cluster Autoscaler or Karpenter?

CLUSTER AUTOSCALER

Scales node groups

- Adjusts Auto Scaling group sizes
- Usually one instance type per group
- Many groups for many shapes
- Does not repack running pods

KARPENTER

Launches right-sized nodes

- Calls EC2 directly for each need
- Chooses across many instance types
- Consolidates under-used nodes
- Diversifies Spot across families

THE TIEBREAKER

Slow scale-out, poor bin-packing or a sprawl of node groups points to Karpenter, or Auto Mode to have it managed.

ECS service auto scaling

- **Application Auto Scaling** adjusts the service's desired task count.
- **Target tracking** on CPU, memory or ALBRequestCountPerTarget.
- **Step, scheduled and predictive** policies for known patterns.
- **CPU is the wrong signal** for I/O-bound services that wait on downstream calls.
- **Capacity providers** make EC2 capacity follow the tasks.

> ON THE EXAM
>
> Scale on the metric that tracks the bottleneck. For request-driven services that is requests per target.

Lambda concurrency

- **Account limit** is shared by every function in the Region, unless reserved.
- **Scaling rate:** 1,000 execution environments per function every 10 seconds.
- **Reserved concurrency** guarantees capacity to a function and caps it. No charge.
- **Provisioned concurrency** keeps environments initialised. Billed while configured.
- **SnapStart** cuts cold starts for Java, Python and .NET without provisioning.

> ON THE EXAM
>
> Reserved concurrency is about who gets capacity. Provisioned concurrency is about cold-start latency.

Your registry is part of your **attack surface**. What you scanned at push time can become vulnerable next week.

Amazon ECR controls that appear in questions

CONTROL	WHAT IT DOES
Basic scanning	OS package CVEs, on push or manual. AWS-native engine
Enhanced scanning	Amazon Inspector, OS and language packages, continuous
Tag immutability	Tags cannot be overwritten; exclusions since July 2025
Lifecycle policies	Expire or archive by age, count or last pull
Archive storage class	Since November 2025; restore within about 20 minutes
Replication	Copy images across Regions and accounts automatically

ON THE EXAM

Continuous rescanning of deployed images requires enhanced scanning. Basic scanning never rescans on its own.

Basic or enhanced scanning?

BASIC SCANNING

Point-in-time OS checks

- Operating system packages only
- Scan on push or on request
- No extra service needed
- Results for that moment only

ENHANCED SCANNING

Continuous, via Inspector

- OS and programming language packages
- Rescans as new CVEs are published
- Findings flow to Security Hub
- Billed through Amazon Inspector

THE TIEBREAKER

Language libraries such as Python or Java packages, or rescanning when new CVEs appear: enhanced.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
Existing Helm charts or operators	EKS, not an ECS rewrite
GPU containers, no host patching	ECS Managed Instances
Pods stay Pending after scale-out	Node-layer scaling is missing or slow
CPU low but latency rising	Wrong target-tracking metric
One function starves another	Reserved concurrency
New CVEs in running images	Enhanced, continuous scanning

ON THE EXAM

Platform, scaling layer, scaling metric, concurrency, registry. Name which one the question is about first.

> Meridian Freight is moving 40 microservices from a self-managed Kubernetes cluster in its data centre to AWS. Teams deploy with Helm charts and rely on two Kubernetes operators.
>
> The platform team is three people and wants to stop patching, scaling and replacing worker nodes.

Which solution meets these requirements with the LEAST operational overhead?

A Rewrite the services as ECS task definitions on Fargate

B Amazon EKS with self-managed node groups and Cluster Autoscaler

C Amazon EKS Auto Mode

D Kubernetes installed on EC2 instances with a custom AMI pipeline

◆ PAUSE AND DECIDE

Which solution meets these requirements with the LEAST operational overhead?

A ECS rewrite

Fargate removes hosts, but rewriting 40 services and replacing two operators is large new work.

B EKS, self-managed nodes

Keeps Kubernetes, but the team is still patching and scaling nodes.

C EKS Auto Mode

Keeps Helm and operators; AWS runs node scaling, patching and add-ons.

D Self-installed Kubernetes

Adds control plane operations on top of everything they already do.

THE SIGNAL

Kubernetes tooling keeps it on EKS. Wanting to stop node operations moves it to Auto Mode.

> Meridian Freight's route optimisation service runs containerised inference on ECS and needs NVIDIA GPU instances. It currently runs on ECS with EC2 Auto Scaling groups and a hand-built AMI patching process.
>
> The company wants to stop managing AMIs and host patching while keeping GPU instances.

What should the solutions architect recommend?

A ECS Managed Instances with a capacity provider specifying GPU instance types

B Move the service to ECS on Fargate with GPU-enabled task sizes

C Keep ECS on EC2 and automate AMI builds with a scheduled pipeline

D Package the model as a Lambda container image

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A ECS Managed Instances

AWS provisions and patches the GPU instances you specify.

B Fargate with GPUs

Fargate does not offer GPUs, so this option does not exist.

C Automated AMI pipeline

Still the company's patching process, just automated. The requirement was to stop owning it.

D Lambda container image

Lambda has no GPU support.

THE SIGNAL

GPU rules out Fargate and Lambda. Stop patching rules out EC2 you manage. One option is left.

Meridian Freight's EKS cluster uses Cluster Autoscaler with six managed node groups, each a single instance type. Batch pods wait several minutes in Pending, many nodes run half empty, and Spot interruptions hit whole node groups at once.

The team wants faster scale-out, better packing and more resilient Spot usage, and will keep managing the cluster itself.

What should the solutions architect do?

A. Add more node groups to cover additional instance sizes

B. Raise the Horizontal Pod Autoscaler's minimum replicas

C. Run low-priority placeholder pods to keep spare nodes warm

D. Replace Cluster Autoscaler with Karpenter, using a NodePool that allows many instance families and Spot

◆ PAUSE AND DECIDE

What should the solutions architect do?

A More node groups

More shapes to manage, and still no consolidation of half-empty nodes.

B Higher HPA minimum

Adds pods, not nodes. Makes Pending worse.

C Placeholder pods

Buys headroom by paying for idle nodes. Packing gets worse, not better.

D Karpenter with diverse NodePool

Launches fitting nodes directly, consolidates, spreads Spot across families.

THE SIGNAL

Three symptoms of single-type node groups: slow fit, poor packing, correlated Spot loss. Karpenter fixes all three.

> Meridian Freight's quoting service runs on ECS behind an Application Load Balancer, with target tracking on average CPU at 60 percent. Each request mostly waits on a partner pricing API.
>
> At peak, response times triple while service CPU stays near 25 percent, and the task count never increases.

Which change makes the service scale with demand?

A. Lower the CPU target to 20 percent

B. Target tracking on ALBRequestCountPerTarget

C. A scheduled action that doubles tasks at peak hours

D. Double the CPU and memory of each task

◆ PAUSE AND DECIDE

Which change makes the service scale with demand?

A **Lower CPU target**

Still the wrong signal. Scales on idle CPU noise and over-provisions off-peak.

B **Requests per target**

Tracks the real load on a service that is waiting, not computing.

C **Scheduled scaling**

Helps the known peak only, and guesses at the size.

D **Bigger tasks**

More CPU for a service that is not using its CPU.

THE SIGNAL

Low CPU with rising latency means the service is waiting. Scale on requests, not on processor time.

In one AWS account, a nightly Lambda batch job scales to thousands of concurrent executions. During that window, Meridian Freight's customer-facing checkout function is throttled.

Checkout needs up to 300 concurrent executions. The company wants to fix this as cost-effectively as possible.

Which solution is MOST cost-effective?

A Configure 300 provisioned concurrency on the checkout function

B Request a higher account concurrency limit

C Schedule the batch job for a time with no checkout traffic

D Set reserved concurrency of 300 on checkout and a reserved cap on the batch function

◆ PAUSE AND DECIDE

Which solution is MOST cost-effective?

A. Provisioned concurrency

Also protects checkout, but bills for 300 warm environments around the clock.

B. Higher account limit

The batch job can grow into any new headroom. Nothing isolates checkout.

C. Reschedule the batch

An online checkout has no reliably quiet hour, and nothing enforces it.

D. Reserved concurrency

Guarantees checkout's capacity and caps the batch, at no charge.

THE SIGNAL

Starvation between functions is a reservation problem. Provisioned concurrency also works, and costs money.

> Meridian Freight's security team requires that container images already running in production are re-evaluated whenever new vulnerabilities are published, including vulnerabilities in Python libraries.
>
> Release tags must never be overwritten, but developers need a nightly tag they can move every day.

Which Amazon ECR configuration meets these requirements?

A. Enhanced scanning with continuous scan, and tag immutability with an exclusion for the nightly tag

B. Basic scanning with scan on push, and mutable tags

C. Enhanced scanning on push only, and tag immutability on every tag

D. Basic scanning triggered daily by EventBridge Scheduler, and tag immutability on every tag

◆ PAUSE AND DECIDE

Which Amazon ECR configuration meets these requirements?

A Enhanced continuous, with exclusion

Rescans as CVEs appear, covers language packages, locks releases, frees nightly.

B Basic on push, mutable

No Python coverage, no rescanning, and release tags can be overwritten.

C Enhanced on push, all immutable

No rescanning after push, and the nightly tag can no longer move.

D Basic daily, all immutable

Basic never sees Python libraries, and nightly cannot move.

THE SIGNAL

Language libraries plus re-evaluation means enhanced continuous. A movable tag among fixed ones means an exclusion.

What to carry out of this module

- **Kubernetes tooling means EKS.** Least node work on EKS means Auto Mode.
- **GPU without host patching:** ECS Managed Instances. Fargate has no GPUs.
- **Scale both layers.** Pods or tasks, and the capacity under them.
- **Reserved concurrency** for fairness, free. Provisioned for cold starts, paid.
- **Enhanced continuous scanning** for language packages and new CVEs.

> ON THE EXAM
>
> App Runner and AWS Batch are no longer on the in-scope list. Do not choose them on this exam.